

Design and Implementation of a Thread-Safe Banking System Using Java

Introduction

Concurrent programming is an important aspect of modern software development because many applications must support multiple users performing operations simultaneously. Banking systems are a common example where multiple transactions may occur on the same account at the same time. If concurrency is not managed correctly, shared account data can become corrupted, leading to incorrect balances and unreliable system behaviour.

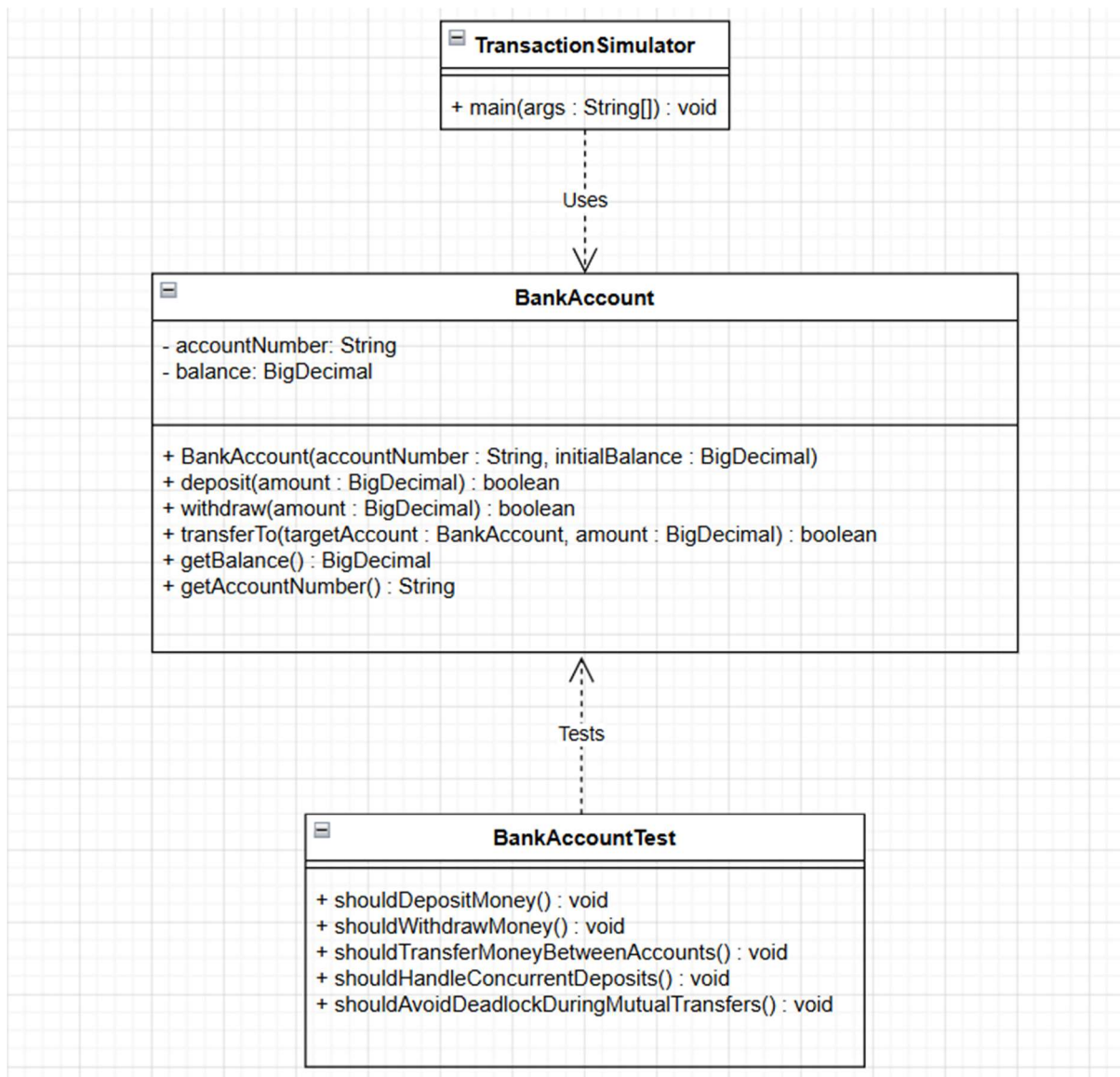
The aim of this assignment is to design and implement a thread-safe banking system in Java that supports concurrent deposits, withdrawals, balance enquiries and transfers between accounts. The system was required to prevent race conditions and deadlocks while maintaining data integrity. The implementation was developed using object-oriented programming principles, Java multithreading and synchronisation techniques. Unit tests were also created to verify the correctness and reliability of the solution.

System Design and Object-Oriented Principles

The solution consists of three classes: BankAccount, TransactionSimulator and BankAccountTest. Figure 1 presents the UML class diagram showing the relationships between these classes. The application was developed using Java 17 in IntelliJ IDEA and tested using JUnit 5.

Figure 1. UML Class Diagram

The UML class diagram below shows the main structure of the thread-safe banking system.



The **BankAccount** class represents the core business object and is responsible for managing account data and transactions. The **Transaction Simulator** class creates multiple threads that perform banking operations concurrently, while the **BankAccountTest** class validates the behaviour of the system through automated testing.

Several object-oriented principles were applied during development. Encapsulation was achieved by declaring the account number and balance as private fields in the **BankAccount** class. This prevents external classes from modifying account data directly and ensures that all updates occur through controlled methods.

```

private final String accountNumber;
private BigDecimal balance;

/**
 * Creates a new bank account with the specified account number
 * and opening balance.
 *
 * @param accountNumber unique identifier for the account
 * @param initialBalance starting account balance
 * @throws IllegalArgumentException if the account number is blank
 *                               or the initial balance is negative
 */
public BankAccount(String accountNumber, BigDecimal initialBalance) {
    if (accountNumber == null || accountNumber.isBlank()) {
        throw new IllegalArgumentException("Account number cannot be empty.");
    }

    if (initialBalance == null || initialBalance.compareTo(BigDecimal.ZERO) < 0) {
        throw new IllegalArgumentException("Initial Balance cannot be negative.");
    }

    this.accountNumber = accountNumber;
    this.balance = initialBalance;
}

```

This design protects the internal state of the object and improves data integrity.

The design also follows the Single Responsibility Principle (SRP), one of the SOLID principles. Each class has a distinct responsibility. The BankAccount class manages account operations, TransactionSimulator handles transaction execution and BankAccountTest focuses on validation and testing. Separating responsibilities improves maintainability and makes the system easier to understand and extend.

Abstraction is also demonstrated through the public methods provided by the BankAccount class. External classes interact with the account through methods such as deposit(), withdraw() and transferTo() without requiring knowledge of the underlying implementation.

Thread Safety and Race Condition Prevention

One of the primary requirements of the assignment was to ensure thread safety. A thread-safe system behaves correctly when multiple threads access shared resources simultaneously.

Without proper synchronisation, race conditions can occur. A race condition occurs when multiple threads attempt to modify shared data at the same time and the final result depends on the order of execution. Schildt and Coward (2024) explain that race conditions are among the most common problems encountered in multithreaded applications because multiple threads may read and modify the same data concurrently.

For example, if two users attempt to deposit money into the same account simultaneously, both threads may read the same balance before either update is applied. This could result in one transaction overwriting the other and producing an incorrect final balance.

To prevent this issue, the BankAccount class uses Java's synchronized keyword. The deposit(), withdraw() and getBalance() methods are declared as synchronized methods.

```
public synchronized boolean deposit(BigDecimal amount)
```

When a thread enters a synchronized method, it acquires the object's monitor lock. Other threads attempting to execute synchronized methods on the same object must wait until the lock is released. This guarantees that only one thread can modify the account balance at a time.

The deposit() method validates the input amount and updates the balance using BigDecimal arithmetic.

```

/**
 * Synchronized to prevent race conditions when updating balance.
 */
public synchronized boolean deposit(BigDecimal amount) { 7 usages
    if (amount == null || amount.compareTo(BigDecimal.ZERO) <= 0) {
        throw new IllegalArgumentException("Deposit Amount must be greater than zero");
    }

    balance = balance.add(amount);
    return true;
}

```

Similarly, the `withdraw()` method verifies that sufficient funds are available before subtracting the requested amount.

```

/**
 * Synchronized to ensure only one thread can withdraw at a time.
 */
public synchronized boolean withdraw(BigDecimal amount) { 5 usages
    if (amount == null || amount.compareTo(BigDecimal.ZERO) <= 0) {
        throw new IllegalArgumentException("Withdraw amount must be greater than zero.");
    }

    if (balance.compareTo(amount) < 0) {
        return false;
    }

    balance = balance.subtract(amount);
    return true;
}

```

Using synchronized methods ensures that balance updates occur safely even when multiple threads access the same account concurrently.

The `getBalance()` method is also synchronized. Although it only reads data, synchronising the method ensures that users receive a consistent balance value while transactions are occurring.

```

/**
 * Synchronized to ensure a consistent balance is returned
 * when accessed by multiple threads.
 */
public synchronized BigDecimal getBalance() { return balance; }

```

Deadlock Prevention

Deadlocks are another significant challenge in concurrent programming. A deadlock occurs when two or more threads wait indefinitely for resources held by each other, preventing execution from continuing.

A common deadlock scenario in banking systems involves transfers between accounts. For example, one thread may attempt to transfer money from Account A to Account B while another simultaneously transfers money from Account B to Account A. If both threads acquire one lock and wait for the other, neither transaction can proceed.

Schildt and Coward (2024) identify deadlocks as a major risk in multithreaded systems and recommend careful lock management to avoid circular waiting conditions.

To prevent deadlocks, the `transferTo()` method acquires locks in a consistent order based on account numbers.

The account with the lower account number is always locked first, followed by the second account.

Because every transfer follows the same locking order, circular dependencies cannot occur. This effectively eliminates the possibility of deadlock while allowing concurrent transfers to be processed safely.

Additional validation was implemented to prevent transfers to the same account and to reject invalid transfer amounts. These checks improve system reliability and demonstrate secure coding practices.

```

BankAccount firstLock;
BankAccount secondLock;

if (this.accountNumber.compareTo(targetAccount.accountNumber) < 0) {
    firstLock = this;
    secondLock = targetAccount;
} else {
    firstLock = targetAccount;
    secondLock = this;
}

synchronized (firstLock) {
    synchronized (secondLock) {
        if (this.balance.compareTo(amount) < 0) {
            return false;
        }

        this.balance = this.balance.subtract(amount);
        targetAccount.balance = targetAccount.balance.add(amount);
        return true;
    }
}

```

Transaction Simulation

The system models a simple banking environment in which a customer account is accessed concurrently by multiple banking services. Transactions such as salary deposits, utility bill payments, mortgage payments, balance enquiries and transfers are processed simultaneously using separate threads. The objective is to maintain accurate account balances while preventing race conditions and deadlocks.

To support safe concurrent transfers, the implementation acquires locks in a consistent account-number order before modifying either account balance. This prevents deadlock situations where two transfer operations could otherwise wait indefinitely for each other's resources. As a result, the simulation demonstrates both thread safety and data integrity while modelling realistic banking transactions.

Each transaction is executed within a separate thread using lambda expressions.

```
Thread salaryPayment = new Thread(() -> {  
    System.out.println("Salary deposited: "  
        + johnAccount.deposit(new BigDecimal( val: "1200.00")));  
    System.out.println("Account Balance after salary deposit is: £"  
        + johnAccount.getBalance());  
});
```

Lambda expressions provide a concise way to implement Runnable behaviour and improve code readability.

All threads are started using the start() method.

```
salaryPayment.start();  
gasBill.start();  
electricityBill.start();  
transferToFriend.start();  
balanceCheck.start();  
mortgage.start();
```

When start() is invoked, Java creates a separate execution path that runs independently of the main thread. This allows multiple transactions to occur concurrently.

The simulator also uses the join() method, as shown in the code snippet below.

```
salaryPayment.join();  
gasBill.join();  
electricityBill.join();  
transferToFriend.join();  
balanceCheck.join();  
mortgage.join();
```


The `join()` method causes the main thread to wait until the specified worker thread completes execution. This ensures that all transactions finish before the program terminates and allows final account balances to be displayed accurately.

Figure 2. Transaction Simulator Output

```
> Task :com.threadsafe.TransactionSimulator.main()

John checked balance: £1870.00
Mortgage payment: Unable to process withdrawal. Insufficient funds.
Salary deposited: true
Account Balance after salary deposit is: £1870.00
Gas bill paid: true
Gas bill £5 discount deposited: true
Electricity bill paid: true
Account Balance after gas bill paid and discount applied is: £1875.00
Electricity bill 5% discount deposited: true
Account Balance after electricity bill and discount applied is: £1879.00
Account Balance is: £1879.00
Transfer John -> Friend: true
John Balance: £1879.00; Friend Balance: £300.00
Transfer Friend -> John: true
John Balance: £1889.00; Friend Balance: £290.00
```

The simulator output demonstrates that multiple transactions can be executed concurrently without causing race conditions or inconsistent balances.

Testing and Validation

Comprehensive unit testing was performed using JUnit 5 to verify system correctness and thread safety.

The following scenarios were tested:

- Successful deposits
- Successful withdrawals
- Insufficient funds withdrawals
- Successful transfers

- Rejected transfers
- Concurrent deposits
- Deadlock prevention

The concurrent deposit test creates three separate threads that deposit funds into the same account simultaneously.

After all threads complete execution using `join()`, the final balance is verified.

```
@Test
void shouldHandleConcurrentDeposits() throws InterruptedException {
    BankAccount account = new BankAccount( accountNumber: "ACC001", BigDecimal.ZERO);

    Thread t1 = new Thread(() -> account.deposit(new BigDecimal( val: "100.00")));
    Thread t2 = new Thread(() -> account.deposit(new BigDecimal( val: "100.00")));
    Thread t3 = new Thread(() -> account.deposit(new BigDecimal( val: "100.00")));

    t1.start();
    t2.start();
    t3.start();

    t1.join();
    t2.join();
    t3.join();

    assertEquals(new BigDecimal( val: "300.00"), account.getBalance());
}
```

This confirms that synchronisation successfully prevents race conditions.

A second concurrency test validates deadlock prevention by creating simultaneous transfers between two accounts.

The test verifies that both threads complete execution successfully and that neither remains blocked. The resulting balances are then checked to ensure that transfers were processed correctly.

The use of `BigDecimal` improves accuracy when handling monetary values and demonstrates best practices for financial calculations. Input validation within constructors and transaction methods further improves reliability and security.

The design follows established object-oriented principles including encapsulation, abstraction and the Single Responsibility Principle. These principles improve maintainability and make the code easier to understand and extend.

One potential enhancement would be the use of `ReentrantLock` from the `java.util.concurrent` package. `ReentrantLock` provides additional features such as timed lock acquisition and improved flexibility. However, the `synchronized` keyword was selected because it provides a simpler and highly readable solution appropriate for the scope of this assignment.

Future enhancements could include transaction history, database persistence, account repositories and additional security controls.

Conclusion

This assignment successfully demonstrated the design and implementation of a thread-safe banking system using Java. The solution combines object-oriented programming principles with multithreading and synchronisation techniques to support safe concurrent access to shared account data.

Race conditions were prevented through synchronized methods, while deadlocks were avoided through consistent lock ordering during transfers. Comprehensive testing verified that the system behaves correctly under concurrent workloads and maintains accurate account balances.

Overall, the project provides a practical demonstration of advanced object-oriented design, secure coding practices and concurrent programming techniques within a realistic banking scenario.

References

Oracle (2026) *Java Platform, Standard Edition Documentation*. Available at:
<https://docs.oracle.com/en/java/javase/> (Accessed: 5 June 2026).

Schildt, H. and Coward, D. (2024) *Java: The Complete Reference, 13th Edition*.
McGraw-Hill.

JUnit (2026) *JUnit User Guide*. Available at:
<https://docs.junit.org/5.14.4/overview.html> (Accessed: 5 June 2026).